PROJECT 3:
Connect: Design for Living Systems

Ziqi Zhou
DESN22848 Interaction Design: Media, Motion and the Body
Professor: Steve Hudak
April 16th, 2026

Table of contents

01 | Design Statement

02 | Research

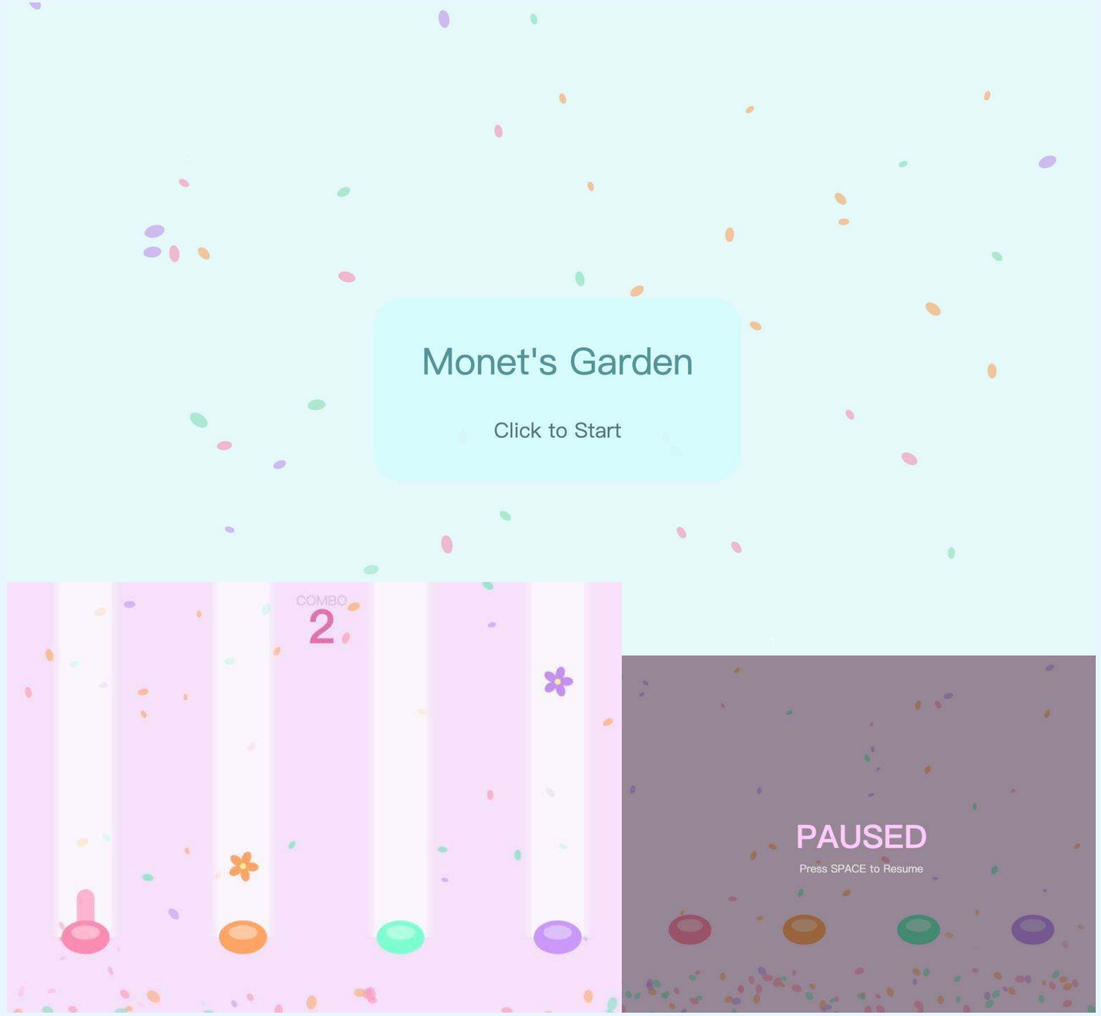
03 | Formstorming

04 | Reflection

05 | Citations

06 | Appendix (WEEK10-13)

Design Statement: Concept

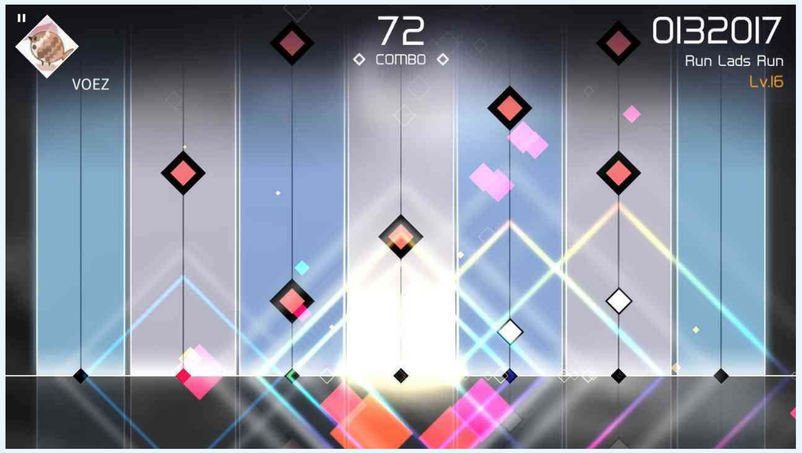


LINK: https://zhou37.phoenix.sheridanc.on.ca/P3.Final_Project_3_Design/

Concept:
In this project, I created a fun little game using Makey Makey and p5.js. The user can control the buttons on the screen by touching a physical object connected to Makey Makey. Trigger the sound effect by hitting each object exactly at a time.

Controls:
Players use the touch panel to play. The Makey Makey is used to control the arrow keys on the keyboard. Each key corresponds to a specific direction on the touch panel. When a note aligns with the target area, touching the correct spot triggers a sound effect and increases the combo count. The game's difficulty increases as the combo count rises.

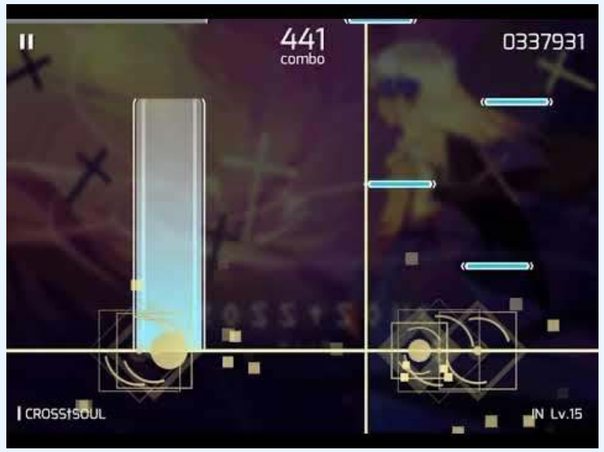
Research: Game Research (Common ground)



Game: Arcaea fandom wiki



Game: Cytus II



Game: Cross†Soul

Common ground:

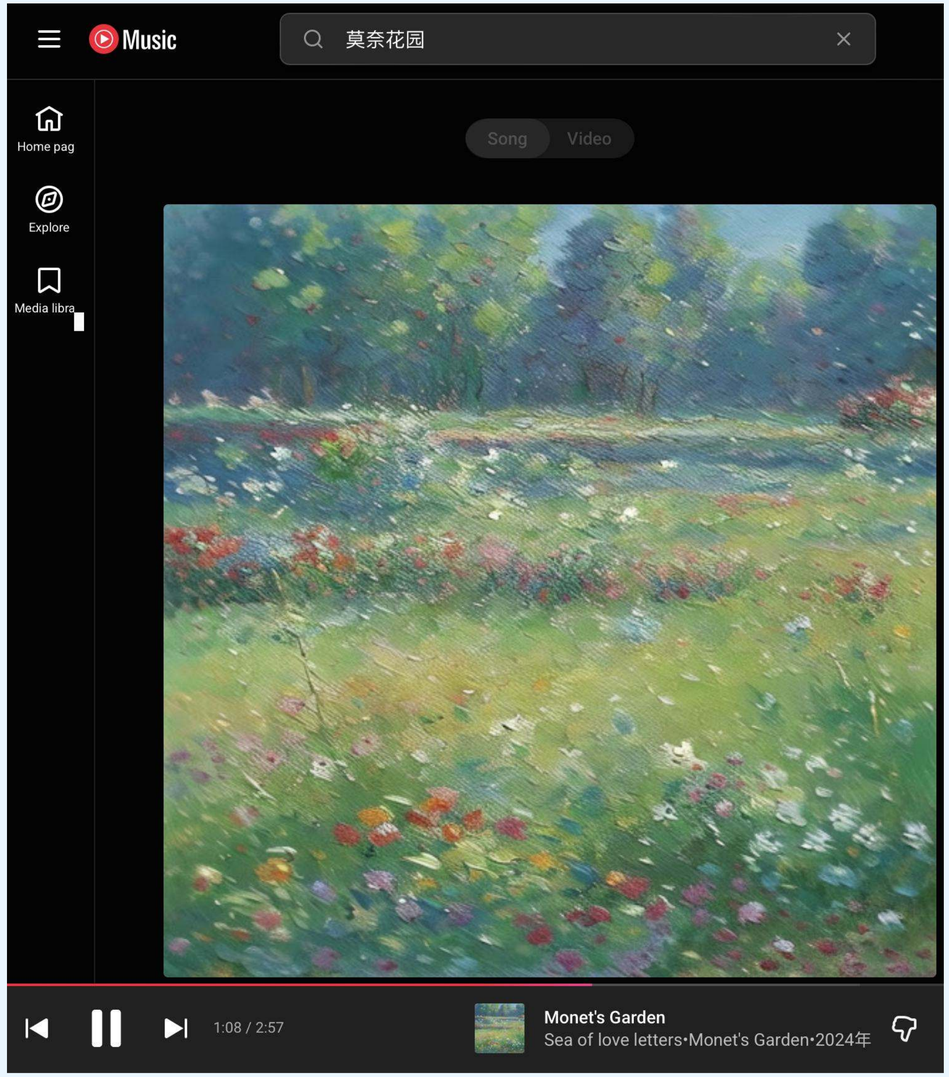
- Judgment Zone: There is a clearly defined “judgment position.”
- Note Commands: Common note types include:

Tap (Single Tap): A quick tap.

Hold (Long Press): Corresponds to sustained notes or syncopated beats in the song.

- Synchronization: Player input must align closely with the music's accents, drum beats, or melodic lines.
- Visual Feedback: Particle effects when hitting notes, a pulsating display of the combo count, and background color changes corresponding to ratings (Perfect/Great).
- Auditory Feedback: The sound of a beat being struck and the sound of a note being cut off upon successful hits.
- Rating System: Uses a multi-tiered rating system such as Miss/Good/Perfect.
- Combo Mechanism: Encourages players to maintain a flawless streak; the higher the combo, the faster the tempo.

Visual Research: Music



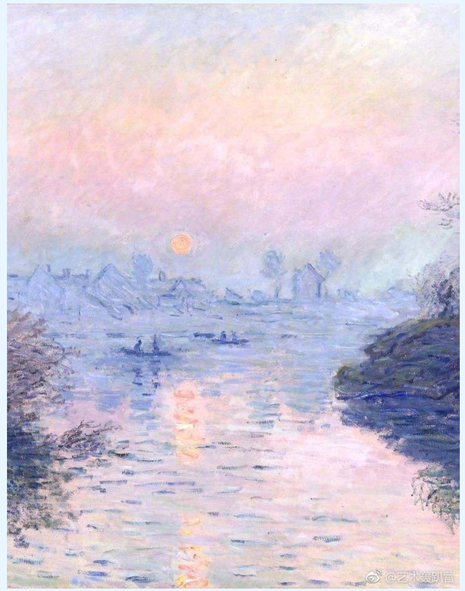
Note: The person pictured is not the original creator. This music is reposted content.

Monet's Garden is an original piano music composed by independent musician Pan Jia-jie. It is included in her album Monet's Garden of the same name.

The artistic features of this work are as follows:

- Inspiration: The title and artistic conception are a direct tribute to the French impressionist painter Claude Monet, inspired by his famous home in Giverny, the Garden of Monet. The garden was the source of inspiration for Monet's famous works such as the Water Lily series and the Nihonbashi series.
- Music style: As a representative style of Pan Jiajie, this piano music is melodious and quiet, with a dreamy and aesthetic color. It tries to use music to depict the changes of light and shadow and the garden scene, so that the listener can feel the poetic atmosphere like entering the Impressionist painting.
- Emotional expression: The song conveys a nostalgia and pursuit of beautiful moments, full of romance and light nostalgia.

Visual Research: Source of inspiration





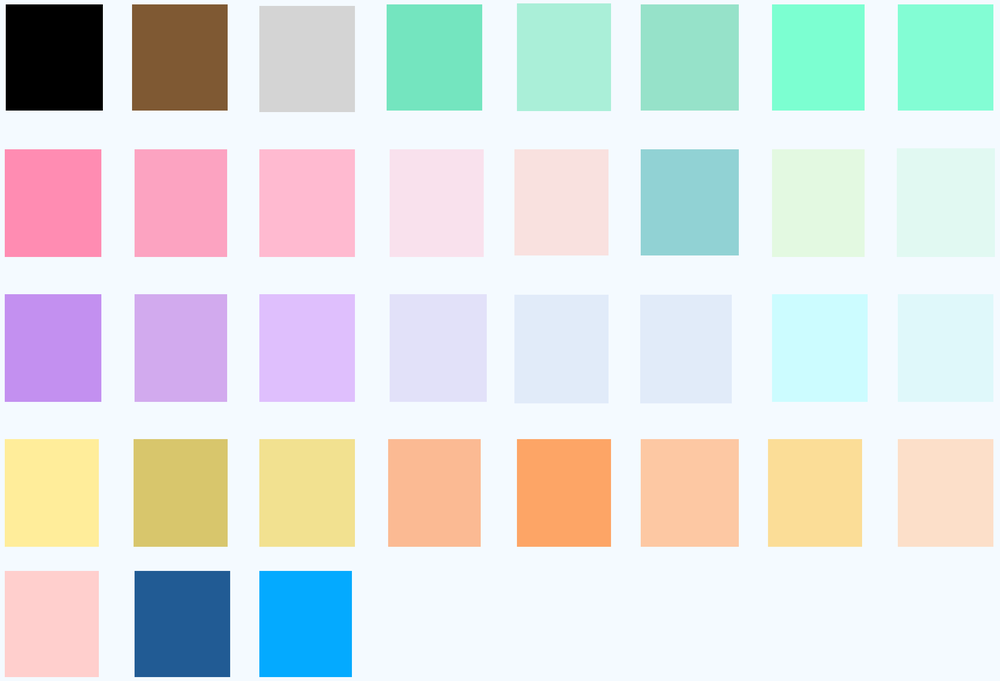




Monet Gardens is the former residence of Claude Monet, the famous French painter. Located in Giverny, France, it is divided into two parts: water garden and garden. The garden, also known as the Normandy Garden, is planted with various flowers and presents a beautiful landscape with changing seasons. The Water Garden is an artificial lake filled with water lilies, where Monet created his famous Water Lilies series and Nihonbashi series.

Feedback: I used the color of the corresponding painting to show it according to the musical inspiration. Also during the game, the background color changes with the music, just like the four seasons.

Visual Research: Color and Font

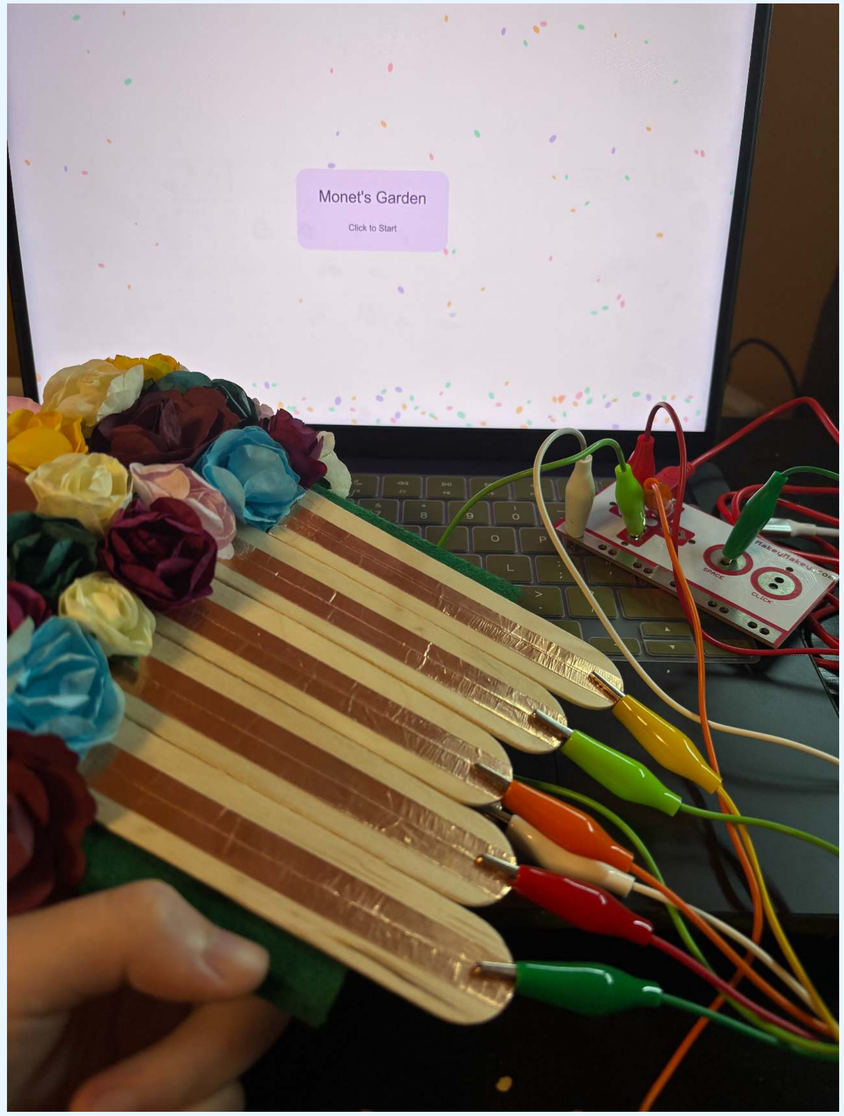


Font Font

Font Font

Font

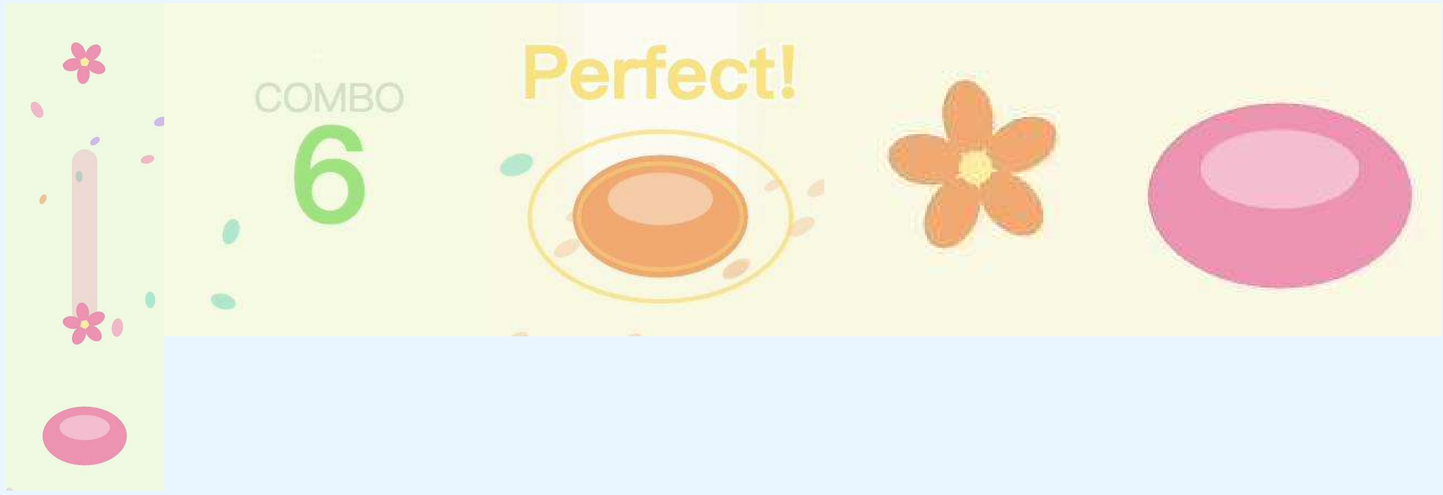
Research: Control board design: Material, technology, operation



This project builds directly on the concept of “physical computing” introduced in Week 1: simple circuits can link physical actions to digital responses. Using a Makey Makey, you can close a circuit by touching the control board to send keyboard signals to p5.js.

Materials:

- **Aluminum foil strip: Main conductive material.**
- **Makey Makey: Connects the control panel.**
- **Flowers: Decorative elements.**
- **Wooden board: Used as piano keys.**



Research: Control board design: Material, Technology, Operation

```
<script
src="https://cdnjs.cloudflare.com/ajax/libs/
p5.js/1.4.0/p5.min.js"></script>

<script
src="https://cdnjs.cloudflare.com/ajax/libs/
p5.js/1.4.0/addons/p5.sound.min.js">
</script>
```

Technology:

- **Makey Makey: Closes the circuit by touching the touchpad, sending key press signals to the computer.**
- **p5.js + p5.sound library: Receives keyboard events, updates game logic, draws falling notes, plays audio, and changes the background color using the HSB color mode.**

```
function draw() {
  globalHueShift = (globalHueShift + 0.05) %
  360;
  let tx = random(-shakeAmount, shakeAmount);
  let ty = random(-shakeAmount, shakeAmount);

  push();
  translate(tx, ty);
  background(globalHueShift, 10, 98);

  updateAmbientPetals();
  updateSparkles();
  updateDecayingPetals();

  if (currentState === GAME_STATE.TITLE) {
    drawTitlePage();
  } else if (currentState ===
  GAME_STATE.ANIMATING) {
    drawAnimatingTransition();
  } else if (currentState ===
  GAME_STATE.PLAYING || currentState ===
  GAME_STATE.PAUSED) {
    drawGamePlay();
    if (currentState === GAME_STATE.PAUSED)
    drawPauseOverlay();
  }

  pop();
  shakeAmount *= 0.7;
}
```

```
function mousePressed() {
  if (currentState === GAME_STATE.TITLE)
  handleStartAction();
}

function drawGamePlay() {
  for (let p of permanentGroundPetals) p.display(this);
  drawAmbientPetals(this);
  for (let k of keys) k.drawHighlight(this);

  if (currentState === GAME_STATE.PLAYING) {
    if (frameCount % spawnRate === 0) {
      let l = floor(random(4));
      notes.push(new Note(lanes[l], l));
    }
    for (let k of keys) k.update();
  }

  for (let i = notes.length - 1; i >= 0; i--) {
    let n = notes[i];
    if (currentState === GAME_STATE.PLAYING) n.update();
    n.display(this);
    if (n.isLong && n.isHolding && currentState ===
    GAME_STATE.PLAYING) {
      if (frameCount % 5 === 0) {
        keys[n.laneIndex].burst(n.col, 2);
        if (frameCount % 15 === 0) combo++;
      }
    }
    if (n.isDead) notes.splice(i, 1);
  }

  if (currentState === GAME_STATE.PLAYING) {
    for (let i = ripples.length - 1; i >= 0; i--) {
    ripples[i].update(); if (ripples[i].isDead)
    ripples.splice(i, 1); }
    for (let i = floatingTexts.length - 1; i >= 0; i--) {
    floatingTexts[i].update(); if
    (floatingTexts[i].isDead) floatingTexts.splice(i, 1);
    }
  }
```

```
class Note {
  constructor(x, laneIndex) {
    this.x = x; this.laneIndex = laneIndex; this.y = -50;
    this.col = MACARON_PALETTE[laneIndex];
    this.isLong = random() > 0.7;
    this.initialLen = this.isLong ? random(150, 350) : 0;
    this.len = this.initialLen;
    this.isHit = false; this.isHolding = false;
    this.isDead = false;
    this.rot = random(TWO_PI);
  }
  update() {
    this.y += noteSpeed; this.rot += 0.02;
    if (this.isLong && this.isHolding) {
      this.y = judgeY; this.len -= noteSpeed * 1.2;
      if (this.len <= 0) {
        floatingTexts.push(new JudgementText("Perfect!",
        this.x, judgeY - 40));
        ripples.push(new GrowthRipple(this.x, judgeY));
        this.isDead = true;
      }
    }
    if (!this.isHit && this.y > judgeY + 60) this.miss();
    else if (this.isLong && this.isHit && !this.isHolding
    && this.len > 0) this.miss();
  }
  miss() {
    if (this.isDead) return;
    floatingTexts.push(new JudgementText("Miss", this.x,
    judgeY));
    combo = 0; this.shatter(); this.isDead = true;
  }
  display(target) {
    target.push();
    if (this.isLong && this.len > 0) {
      target.noStroke();
      target.fill(this.col.h, this.col.s, this.col.b,
      this.isHolding ? 60 : 30);
      target.rect(this.x - 12, this.y - this.len, 24,
      this.len, 12);
    }
```

```
  drawHighlight(target) {
    if (this.isPressed || this.highlightAlpha > 0) {
      let alpha = this.isPressed ? 40 :
      this.highlightAlpha;
      target.push(); target.noStroke();
      for(let i=0; i<6; i++) { let w = 70 + i * 5;
      target.fill(0, 0, 100, (alpha / (i + 1)));
      target.rect(this.x - w/2, 0, w, this.y); }
      target.pop();
    }
  }
  display(target) {
    target.noStroke();
    for (let p of this.particles) { target.fill(p.col.h,
    p.col.s, p.col.b, p.alpha); target.push();
    target.translate(p.x, p.y); target.rotate(p.alpha *
    0.1); target.ellipse(0, 0, p.size, p.size * 0.5);
    target.pop(); }
    target.push(); target.translate(this.x, this.y);
    let s = this.isPressed ? 65 : 80;
    target.fill(this.col.h, this.col.s, this.isPressed ?
    100 : this.col.b);
    target.ellipse(0, 0, s, s * 0.7);
    target.fill(0, 0, 100, 40); target.ellipse(0, -s*0.1,
    s * 0.6, s * 0.3);
    target.pop();
  }
  burst(noteColor, count) {
    let burstCol = noteColor || this.col;
    for (let i = 0; i < count; i++) {
      let angle = random(TWO_PI); let spd = random(2, 6);
      this.particles.push({ x: this.x, y: this.y, vx:
      cos(angle)*spd, vy: sin(angle)*spd, alpha: 100,
      size: random(8, 15), col: burstCol });
      if (count > 5) decayingBurstPetals.push(new
      DecayingPetal(this.x + random(-50, 50), height -
      random(5, 50), burstCol));
    }
```

Research: Control board design: Material, Technology, Operation

```
function drawAnimatingTransition() {
  gameGraphics.background(globalHueShift, 10, 98);
  for (let p of permanentGroundPetals)
  p.display(gameGraphics);
  drawGamePlay();
  doorOffset += doorSpeed;
  image(gameGraphics, -doorOffset, 0, width/2, height, 0
  0, width/2, height);
  image(gameGraphics, width/2 + doorOffset, 0, width/2,
  height, width/2, 0, width/2, height);
  if (doorOffset > width/2) currentState =
  GAME_STATE.PLAYING;
}

function initializeGameLogic() {
  notes = []; keys = []; permanentGroundPetals = [];
  decayingBurstPetals = []; floatingTexts = []; sparkles
  = []; ripples = []; judgeY = height * 0.75;
  let laneWidth = width / 5; lanes = [width/2 -
  laneWidth*1.5, width/2 - laneWidth*0.5, width/2 +
  laneWidth*0.5, width/2 + laneWidth*1.5];
  for (let i = 0; i < 4; i++) keys.push(new
  MacaronKey(lanes[i], judgeY, MACARON_PALETTE[i]));
  for (let i=0; i<100; i++)
  permanentGroundPetals.push(new
  GroundPetal(random(width), random(height-100, height),
  random(MACARON_PALETTE)));
}

function getLaneFromKeyCode(code) {
  if (code === LEFT_ARROW) return 0; if (code ===
  UP_ARROW) return 1;
  if (code === DOWN_ARROW) return 2; if (code ===
  RIGHT_ARROW) return 3;
  return -1;
}
```

```
class JudgementText { constructor(content, x, y) {
this.content = content; this.x = x; this.y = y;
this.alpha = 100; this.scale = 0.5; this.upOffset =
this.isDead = false; this.col = (this.content === "
? { h: 0, s: 0, b: 55 } : { h: 48, s: 85, b: 100 };
update() { this.upOffset -= 1.6; this.alpha -= 2.2;
(this.scale < 1.1) this.scale += 0.1; if (this.alph
0) this.isDead = true; } display(target) { target.p
target.translate(this.x, this.y + this.upOffset);
target.scale(this.scale); target.textAlign(CENTER,
CENTER); if (this.content !== "Miss") { target.stro
0, 100, this.alpha * 0.7); target.strokeWeight(3);
target.fill(this.col.h, this.col.s, this.col.b,
this.alpha); target.textSize(28); target.textStyle(
target.text(this.content, 0, 0); target.pop(); } }

function updateSparkles() { if (frameCount % 8 ===
sparkles.push(new Sparkle()); for (let i =
sparkles.length - 1; i >= 0; i--) { sparkles[i].upd
sparkles[i].display(this); if (sparkles[i].isDead)
sparkles.splice(i, 1); } }

function updateDecayingPetals() { for (let i =
decayingBurstPetals.length - 1; i >= 0; i--) {
decayingBurstPetals[i].update(); if
(decayingBurstPetals[i].isDead)
decayingBurstPetals.splice(i, 1); } }

function updateAmbientPetals() { if (frameCount % 6
&& ambientPetals.length < 80) ambientPetals.push(ne
AmbientPetal(random(width), -20)); for (let i =
ambientPetals.length - 1; i >= 0; i--) {
ambientPetals[i].update(); if (ambientPetals[i].y >
height || ambientPetals[i].alpha <= 0)
ambientPetals.splice(i, 1); } }
```

```
class GroundPetal { constructor(x, y, col) { this.x
this.y = y; this.size = random(10, 20); this.angle
random(TWO_PI); this.col = col; this.alpha = 60; }
display(target) { target.push(); target.translate(t
this.y); target.rotate(this.angle); target.noStroke
target.fill(this.col.h, this.col.s - 5, this.col.b,
this.alpha); target.ellipse(0, 0, this.size, this.s
0.7); target.pop(); } }

class DecayingPetal { constructor(x, y, col) { this
x; this.y = y; this.size = random(8, 15); this.angl
random(TWO_PI); this.col = col; this.alpha = 100;
this.life = 150 + random(50); this.isDead = false;
update() { this.life--; if (this.life < 80) this.al
2; if (this.alpha <= 0 || this.life <= 0) this.isDe
true; } display(target) { target.push();
target.translate(this.x, this.y);
target.rotate(this.angle); target.noStroke();
target.fill(this.col.h, this.col.s - 5, this.col.b,
this.alpha); target.ellipse(0, 0, this.size, this.s
0.7); target.pop(); } }

class Sparkle { constructor() { this.x = random(wid
this.y = random(height); this.alpha = 0; this.maxAl
random(40, 80); this.fadeStep = random(1, 2);
this.growing = true; this.size = random(2, 4);
this.isDead = false; this.hue = globalHueShift; }
update() { if (this.growing) { this.alpha += 4; if
(this.alpha >= this.maxAlpha) this.growing = false;
else { this.alpha -= this.fadeStep; if (this.alpha
this.isDead = true; } } display(target) { target.pu
target.noStroke(); target.fill(this.hue, 5, 100,
this.alpha); target.ellipse(this.x, this.y, this.si
target.rectMode(CENTER); target.rect(this.x, this.y
this.size * 3, 0.5); target.rect(this.x, this.y, 0.
this.size * 3); target.pop(); } }
```

```
unction updateAmbientPetals() { if (frameCount % 6 ===
& ambientPetals.length < 80) ambientPetals.push(new
mbientPetal(random(width), -20)); for (let i =
mbientPetals.length - 1; i >= 0; i--) {
mbientPetals[i].update(); if (ambientPetals[i].y >
eight || ambientPetals[i].alpha <= 0)
mbientPetals.splice(i, 1); } }

lass AmbientPetal { constructor(x, y) { this.x = x;
his.y = y; this.size = random(8, 18); this.speed =
andom(1, 3); this.rot = random(TWO_PI); this.swing =
andom(0.5, 2); this.col = random(MACARON_PALETTE);
his.alpha = 80; } update() { this.y += this.speed;
his.x += sin(frameCount * 0.02) * this.swing; if (this.
 height * 0.7) this.alpha -= 1.5; } display(target) {
arget.push(); target.translate(this.x, this.y);
arget.rotate(this.rot); target.noStroke();
arget.fill(this.col.h, this.col.s - 10, this.col.b,
his.alpha); target.ellipse(0, 0, this.size, this.size *
.6); target.pop(); } }

unction drawCombo() { push(); translate(width/2, 100);
extAlign(CENTER, CENTER); noStroke(); let comboHue =
globalHueShift + 40) % 360; fill(comboHue, 20, 80, 50);
extSize(18); text("COMBO", 0, -35); let s = 1 +
in(frameCount * 0.1) * 0.05; scale(s); fill(0, 0, 100,
0); textSize(64); text(combo, 2, 2); fill(comboHue, 50,
0); textStyle(BOLD); text(combo, 0, 0); pop(); }

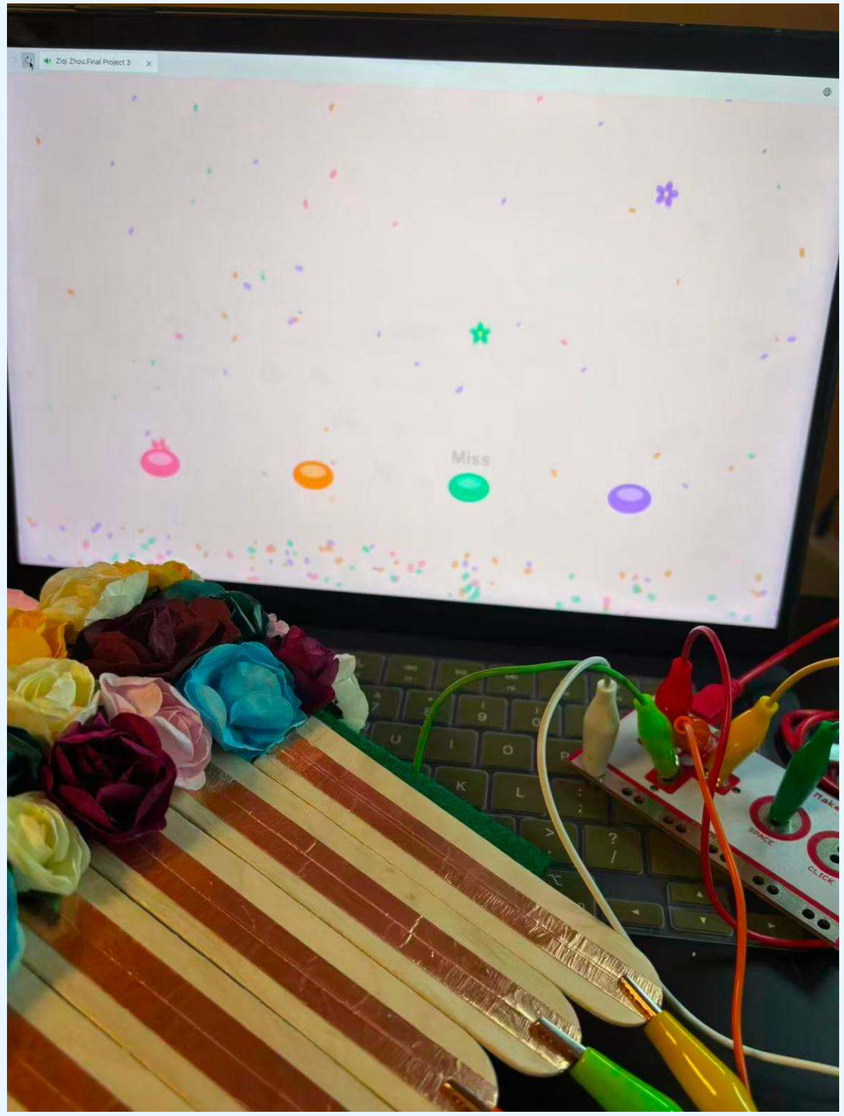
unction drawAmbientPetals(target) { for (let p of
mbientPetals) p.display(target); }

unction windowResized() { resizeCanvas(windowWidth,
indowHeight); gameGraphics = createGraphics(windowWidth
indowHeight); gameGraphics.colorMode(HSB, 360, 100, 100
00); initializeGameLogic(); }
```

Technology:
I used three core functions of p5.js to achieve smooth interactions and animations:

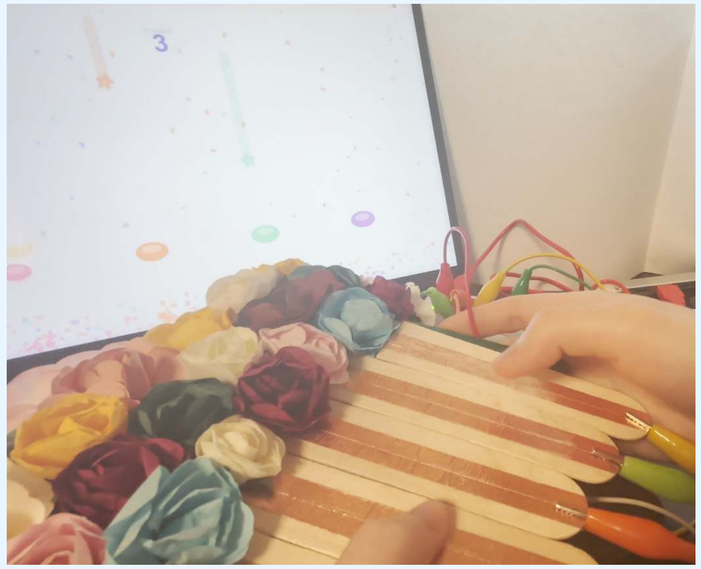
- preload() – Resource preloading. I used “loadSound()” to preload the background music and all sound effects in advance to prevent lag during gameplay.
- setup() – Environment initialization. I created the canvas, set the HSB color mode (to lay the groundwork for the game’s dreamlike color transitions), and initialized the game state.
- draw() – Main loop (approximately 60 frames per second). Continuously erases and redraws all elements on the screen based on the current game state (title screen, animation, in-game, or paused).
- Note class – A note. Manages its own descent position, rotation angle, long-press detection, and “shatter” animation upon being hit.
- MacaronKey Class – The four circular buttons at the bottom. Listens for keyboard input and triggers a particle burst when hit.
- GrowthRipple Class – A golden ripple. Created when a “Perfect” hit is detected, it simulates the spread of water ripples using two elliptical frames, with transparency (alpha) gradually fading.
- HSB Global Color Shift: `globalHueShift` increases slowly with each frame, cycling the background hue between 0 and 360 degrees to achieve a dreamy, Monet-style flow of color.
- Haptic Feedback: When a note is hit, `shakeAmount` increases, causing the main canvas to undergo a small random displacement via `translate(tx, ty)`, enhancing the sense of impact.
- Ambient Particles: AmbientPetal (ambient petals) and Sparkle (sparkles) operate independently of the game logic, continuously drifting to add a sense of “breathing.”

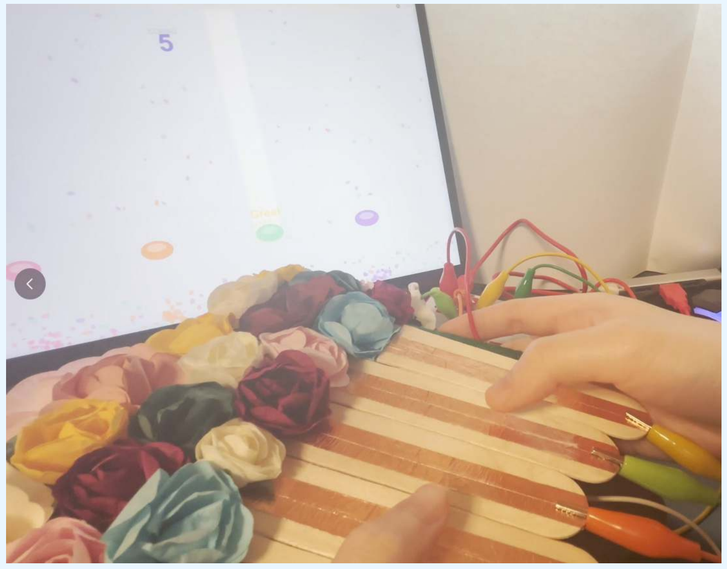
Research: Control board design: Material, Technology, Operation



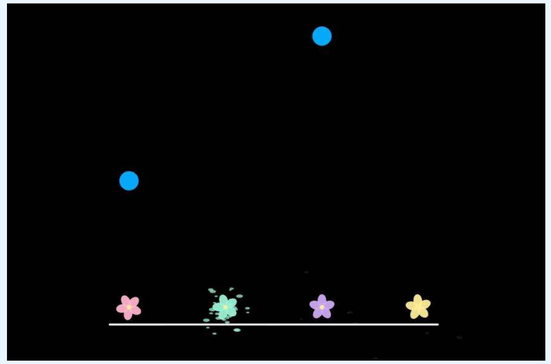
Operation:

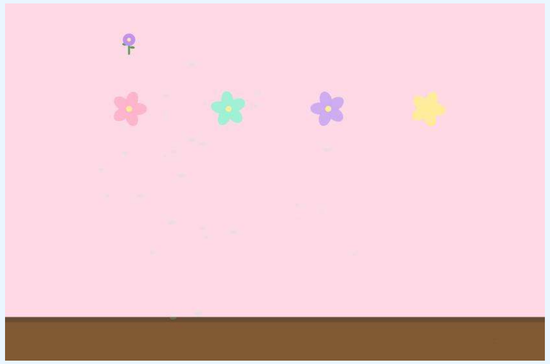
1. Connect the control panel via Makey Makey
2. Hit the corresponding note through the control pad.
3. Score more points per hit. A miss continues the game.
4. Click the space to pause or not.





Research: Work and Iteration



















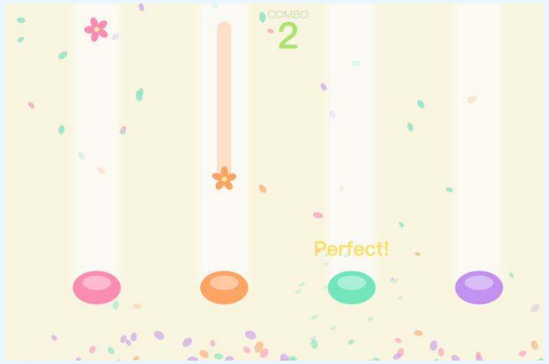
Iteration:

- Determine the game mode
- Assign buttons
- Add bottom petals
- Add checkpoints
- Add visual feedback, such as background petals
- Add a scoring system, such as Miss/Good/Perfect
- Add a combo mechanic, such as a combo counter
- Add note instructions, such as “tap” and “hold”
- Add button backlighting
- Add a pause screen
- Modify the main screen

Research: Work and Iteration







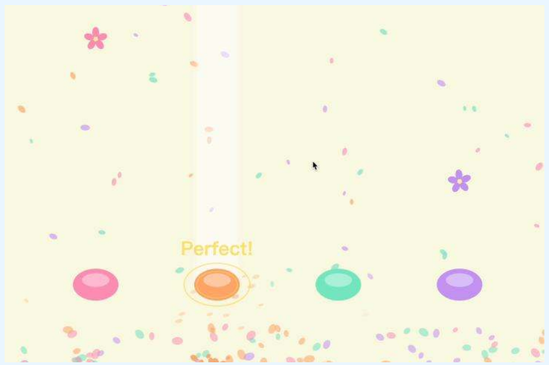








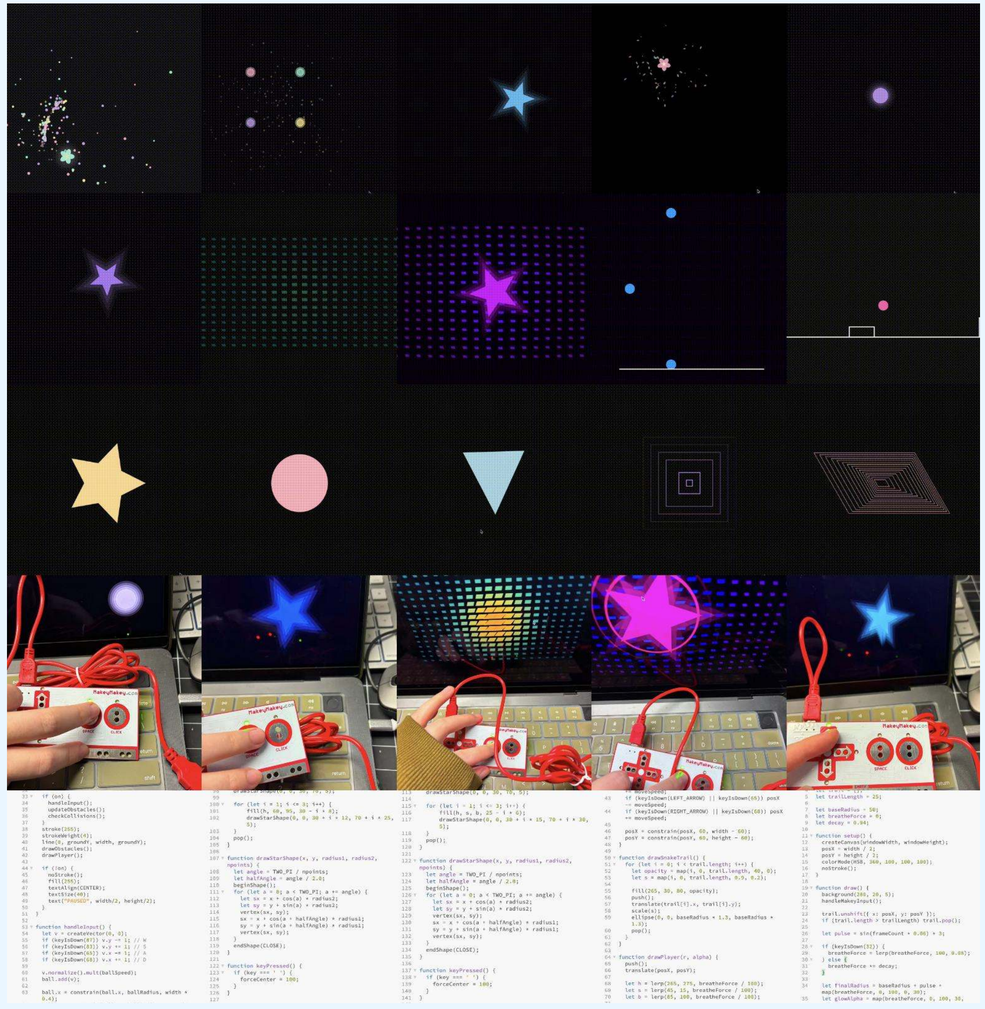




Iteration:

- Determine the game mode
- Assign buttons
- Add bottom petals
- Add checkpoints
- Add visual feedback, such as background petals
- Add a scoring system, such as Miss/Good/Perfect
- Add a combo mechanic, such as a combo counter
- Add note instructions, such as “tap” and “hold”
- Add button backlighting
- Add a pause screen
- Modify the main screen

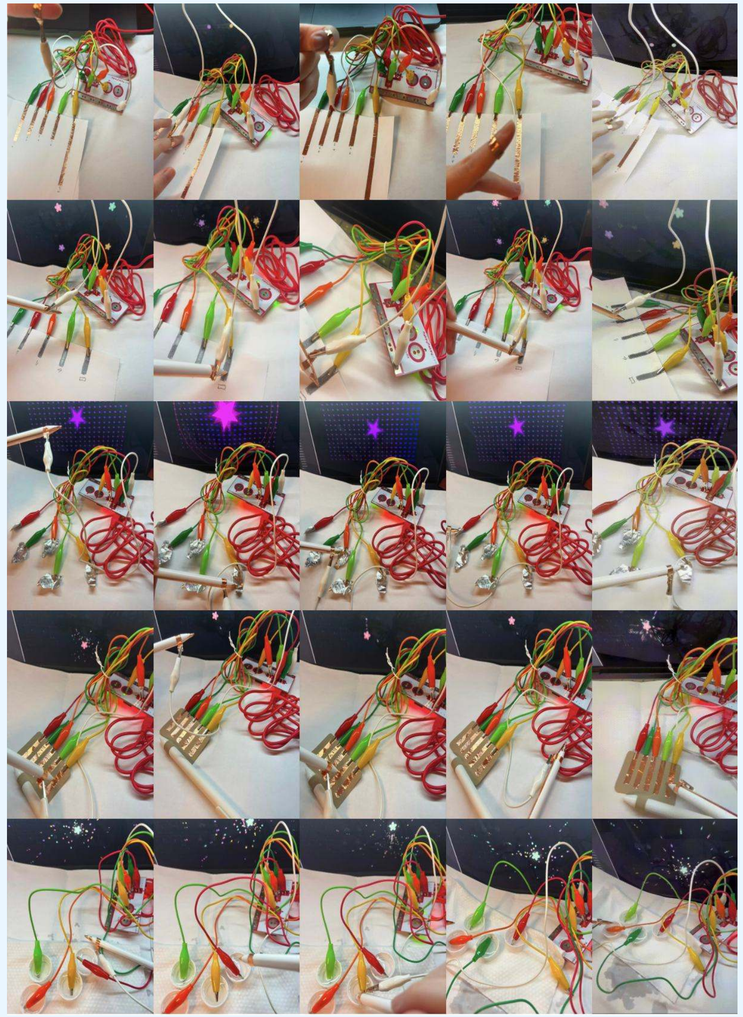
Formstorming:



Feedback:
In Activity 1, I learned how to use p5.js and create new visual effects, as well as how to influence the visual effects through the interaction between sound and MakeyMakey.

LINK: http://zhou37.phoenix.sheridanc.on.ca/DESN22848/MM&B%20Module%203%20Web%20Template/

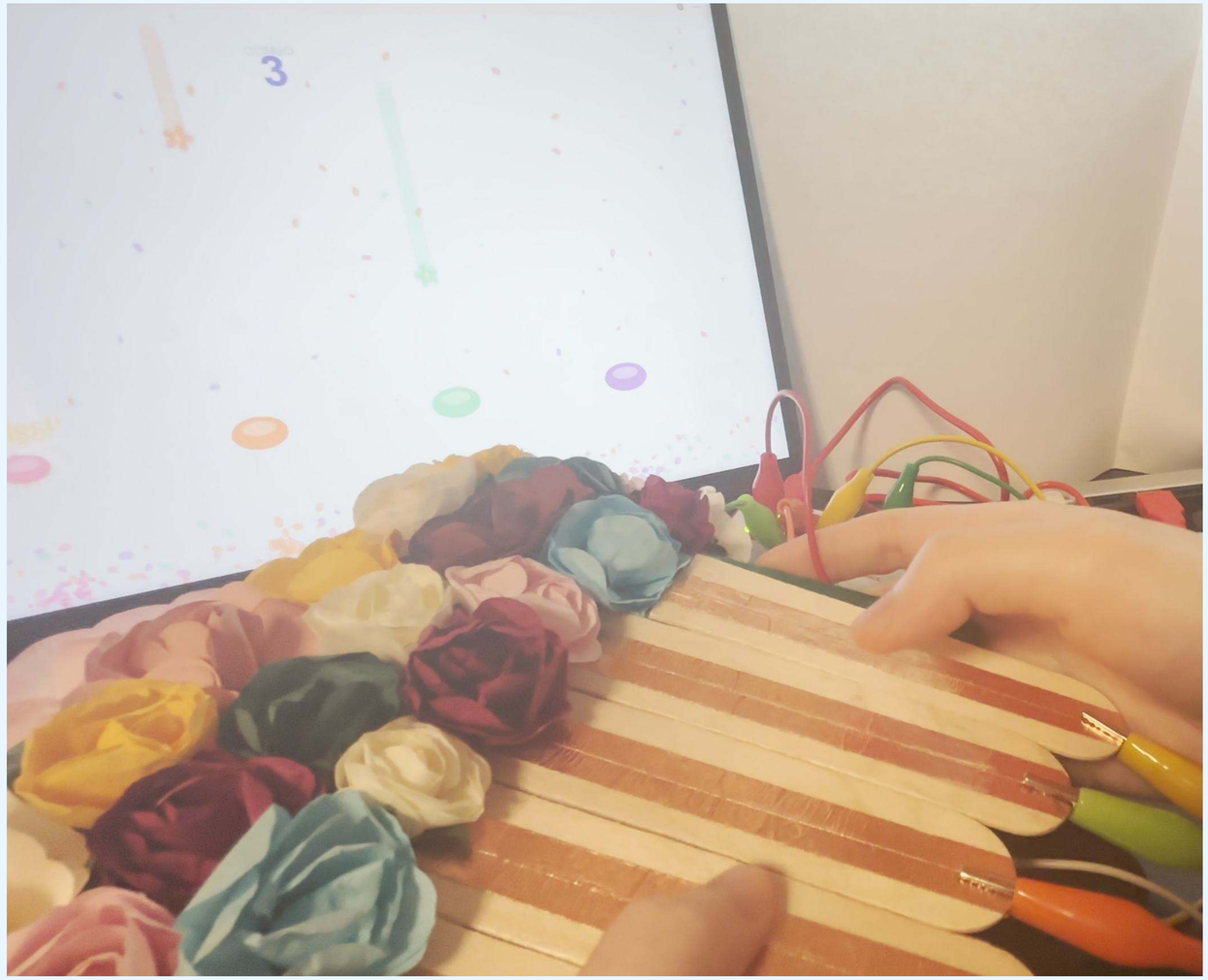
Formstorming:



Feedback:
In Activity 2, I used five conductive materials to test which ones had good electrical conductivity. Can the operation control be carried out through MakeyMakey to change the visual effect?

LINK: http://zhou37.phoenix.sheridanc.on.ca/DESN22848/MM&B%20Module%203%20Web%20Template/

Formstorming: Final Project 3 Design



Feedback:
In this project, I used Makey Makey and p5.js to create a fun audio game, an interactive experience that combines real-world touch with digital animation. The user can control the buttons on the screen by touching a physical object connected to Makey Makey (corresponding to the left, up, down, and right arrow keys).

My inspiration comes from the painting of Monet. A sound effect is triggered when the user accurately hits the falling flower note. I implemented the global dynamic color shift by using p5.js HSB color system. As the game continues to accumulate, the color of each frame of the background will change subtly, creating a dreamy, smart light and shadow effect.

LINK: http://zhou37.phoenix.sheridanc.on.ca/DESN22848/MM&B%20Module%203%20Web%20Template/

Reflection: What I Learned

```
const GAME_STATE = { TITLE: 'TITLE', ANIMATING:
'ANIMATING', PLAYING: 'PLAYING', PAUSED: 'PAUSED' };
let currentState = GAME_STATE.TITLE;

let bgm, hitSfx;

let doorOffset = 0, doorSpeed = 12, gameGraphics;
let ambientPetals = [], floatingTexts = [];
let permanentGroundPetals = [];
let decayingBurstPetals = [];
let sparkles = [], ripples = [];

let lanes = [], keys =
let noteSpeed = 5;
let spawnRate = 50;
let combo = 0;
let shakeAmount = 0;
let globalHueShift = 33

const MACARON_PALETTE =
  { h: 340, s: 45, b: 1
  { h: 160, s: 50, b: 9
];


function preload() {
  bgm = loadSound('musi
  hitSfx = loadSound('s
}
```

```
drawHighlight(target) {
  if (this.isPressed || this.highlightAlpha > 0) {
    let alpha = this.isPressed ? 40 :
    this.highlightAlpha;
    target.push(); target.noStroke();
    for(let i=0; i<6; i++) { let w = 70 + i * 5;
    target.fill(0, 0, 100, (alpha / (i + 1)));
    target.rect(this.x - w/2, 0, w, this.y); }
    target.pop();
  }
}
display(target) {
  target.noStroke();
  for (let p of this.particles) { target.fill(p.col.h,
  p.col.s, p.col.b, p.alpha); target.push();
  target.translate(p.x, p.y); target.rotate(p.alpha *
  0.1); target.ellipse(0, 0, p.size, p.size * 0.5);
  target.pop(); }
  target.push(); target.translate(this.x, this.y);
  let s = this.isPressed ? 65 : 80;
  target.fill(this.col.h, this.col.s, this.isPressed ?
  100 : this.col.b);
  target.ellipse(0, 0, s, s * 0.7);
  target.fill(0, 0, 100, 40); target.ellipse(0, -s*0.1,
  s * 0.6, s * 0.3);
  target.pop();
}
burst(noteColor, count) {
  let burstCol = noteColor || this.col;
  for (let i = 0; i < count; i++) {
    let angle = random(TWO_PI); let spd = random(2, 6);
    this.particles.push({ x: this.x, y: this.y, vx:
    cos(angle)*spd, vy: sin(angle)*spd, alpha: 100,
    size: random(8, 15), col: burstCol });
    if (count > 5) decayingBurstPetals.push(new
    DecayingPetal(this.x + random(-50, 50), height -
    random(5, 50), burstCol));
  }
```

Feedback:
Technically, I learned how to create a smooth, global color gradient for the background by mastering the HSB color mode in p5.js. I also learned how to define four states—Title, Transition, In-Game, and Pause—using “GAME_STATE”, and how to switch between them via the control interface to avoid code clutter.

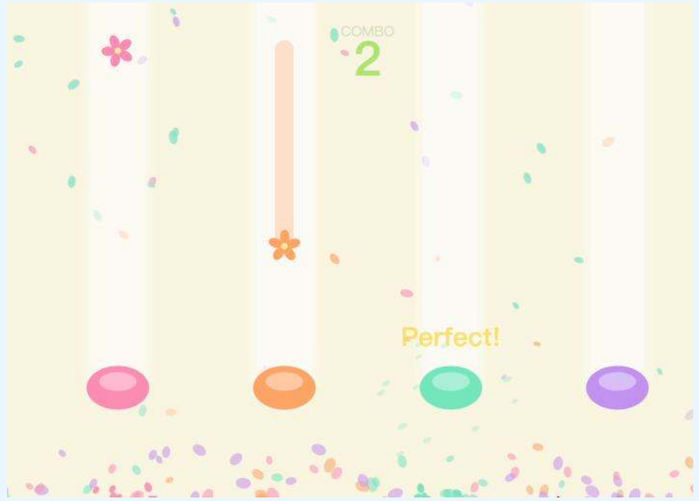








Reflection: Challenges



Challenge 1 – Note Duration (Flower)

- Issue: When rapidly tapping multiple notes in succession, some long-pressed notes experience a slight delay, disrupting the rhythm.
- Solution: Preload all sound files in “preload()” and call “.play()” directly instead of creating a new audio object each time.



Challenge 2 – Jarring Colors

- Issue: In early versions, the colors looked discordant and clashed with the “dreamy” theme I was aiming for.
- Solution: I adopted the HSB color mode in p5.js to achieve smooth color transitions and changes.



Challenge 3 – Insufficient Hit Feedback

- Issue: During user testing, users were unsure whether they had “hit” the notes.
- Solution: Added three types of feedback:

A “petal” animation triggered when a note is clicked (the “burst” method of the “Note” class)
Golden ripples generated upon a successful hit (the “GrowthRipple” class)
A slight vibration of the main canvas (“shakeAmount” + “translate”)

Reflection: Summary, Future Improvements

```
const GAME_STATE = { TITLE: 'TITLE', ANIMATING:
'ANIMATING', PLAYING: 'PLAYING', PAUSED: 'PAUSED' };
let currentState = GAME_STATE.TITLE;

let bgm, hitSfx;

let doorOffset = 0, doorSpeed = 12, gameGraphics;
let ambientPetals = [], floatingTexts = [];
let permanentGroundPetals = [];
let decayingBurstPetals = [];
let sparkles = [], ripples = [];

let lanes = [], keys = [], notes = [], judgeY;
let noteSpeed = 5;
let spawnRate = 50;
let combo = 0;
let shakeAmount = 0;
let globalHueShift = 330;

const MACARON_PALETTE = [
```



Summary:
In this project, I learned

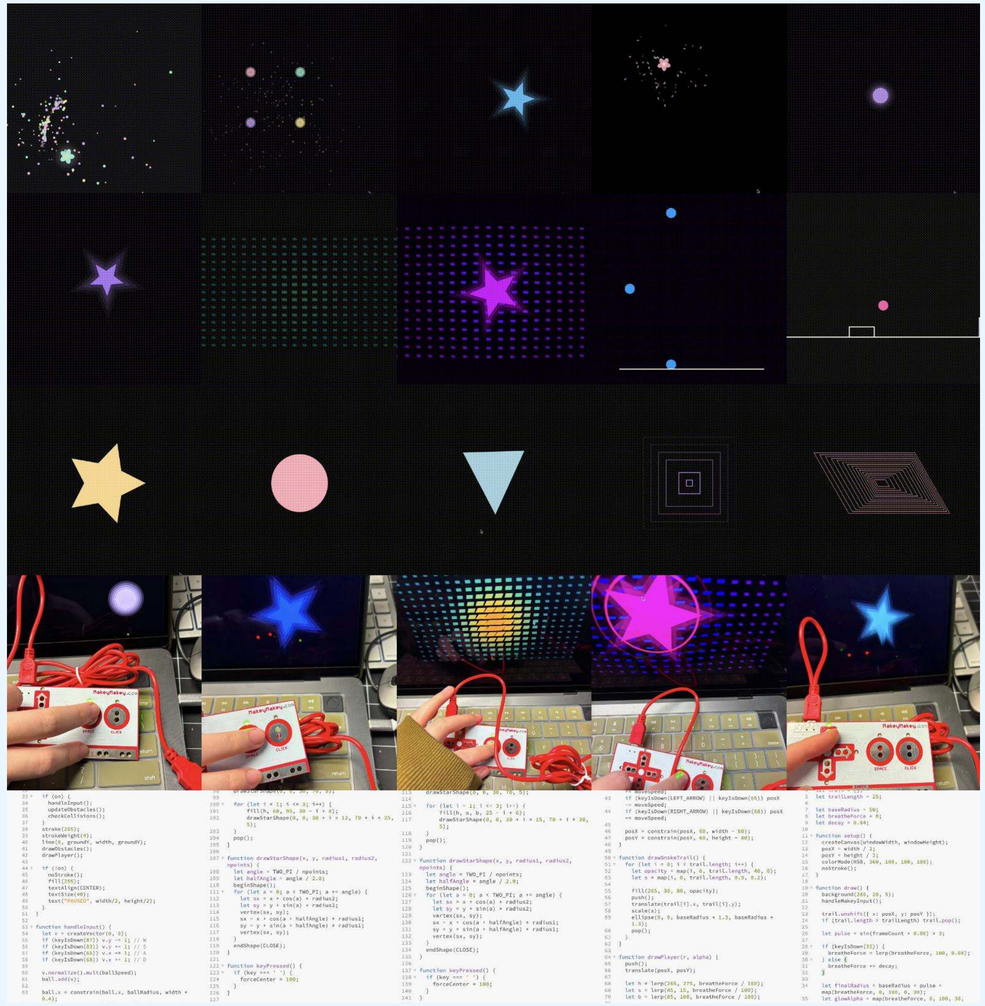
- **new p5.js code (such as “preload,” “setup,” and “draw”), and implemented smooth transition animations and audio preloading.**
- **I learned the HSB color model and used `globalHueShift` to continuously adjust hue values, recreating the dreamlike play of light and shadow found in Monet’s paintings.**
- **I practiced state machine management (e.g., title/transition/ in-game/pause), resulting in a clear and maintainable code structure.**
- **I applied object-oriented programming to design classes such as “Note”, “MacaronKey”, and “GrowthRipple”, ensuring that each visual element operates independently.**

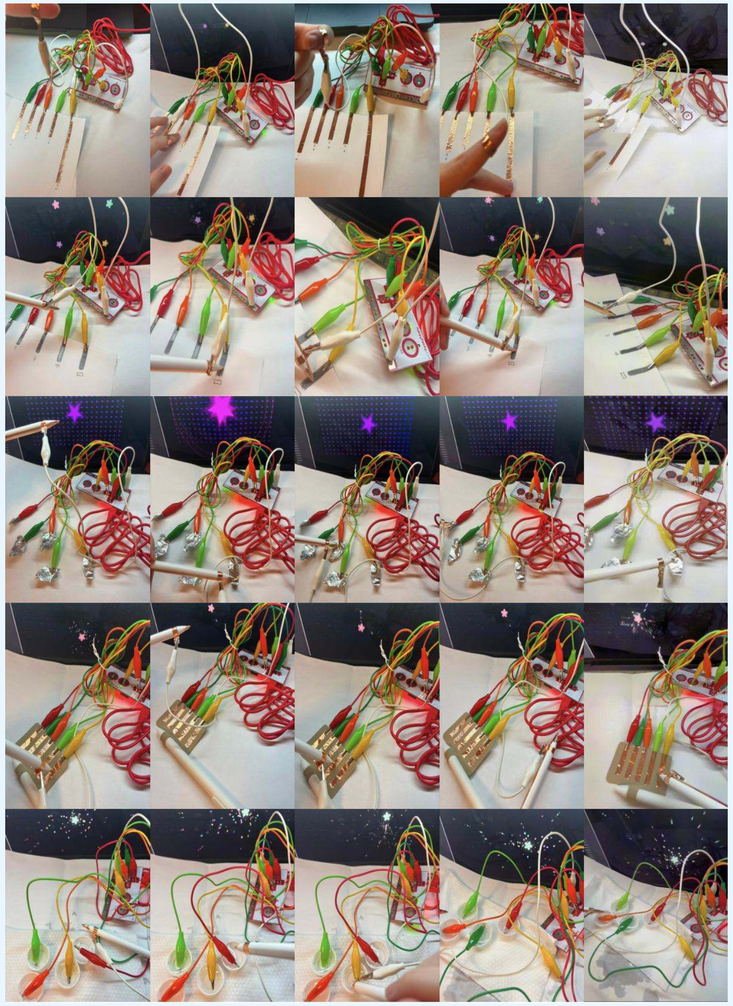
Future Improvements:
If I could make improvements in the future, I would like to add physical LED feedback to the control panel. By leveraging the Makey Makey’s ability to conduct electricity when touched, I could make the LED light up in sync, creating a three-pronged feedback system that combines touch, light, sound, and visuals.

Citations:

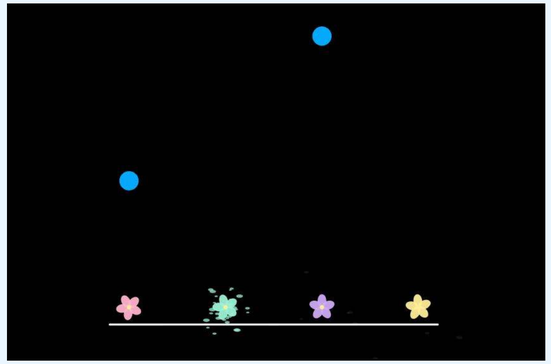
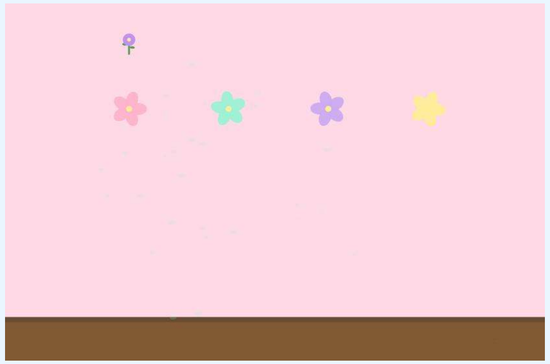
https://webptogif.net/zh/mp4-to-gif
https://www.claudemonetgallery.org/media/paintingsL/80919/irises-in-monets-garden-reprod.jpg?width=600
https://x0.ifengimg.com/res/2020/3249E84F74F7706FA2E58DB881F8489A3E521DA0_size265_w1080_h812.jpeg
https://cdn.arstechnica.net/wp-content/uploads/2017/02/SS041.jpg
https://i.ytimg.com/vi/F91czIA5BIU/maxresdefault.jpg
https://n.sinaimg.cn/sinacn/w741h959/20180306/1ef2-fxipenn9126652.jpg
https://img.freepik.com/photos-premium/peinture-impressionniste-du-jardin-monet-centree-plantes-composition-diagonale-au-premier-plan-o_1097265-98986.jpg?semt=ais_hybrid

Appendix: WEEK10,11-Activity 1,2





Appendix: Week 12,13 – Final Design











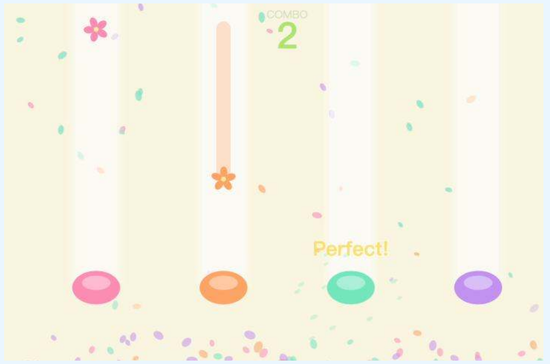





Appendix: Week 12,13 – Final Design





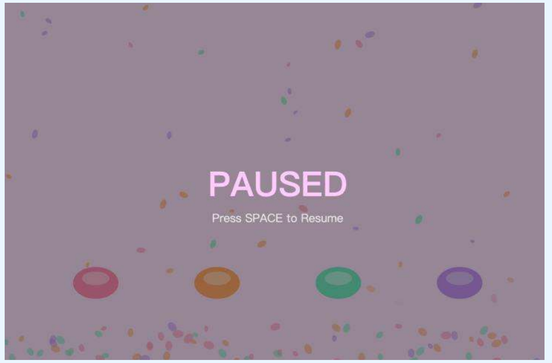




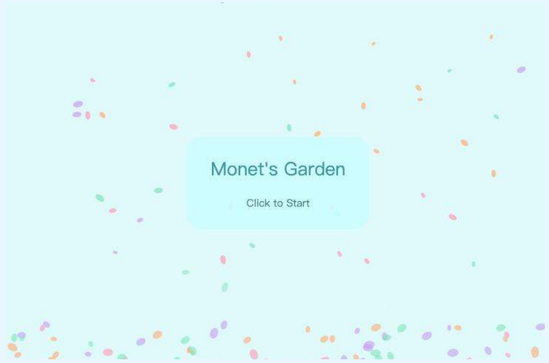
















Appendix: Week 12,13 – Final Design

```
function draw() {
  globalHueShift = (globalHueShift + 0.05) %
  360;
  let tx = random(-shakeAmount, shakeAmount);
  let ty = random(-shakeAmount, shakeAmount);

  push();
  translate(tx, ty);
  background(globalHueShift, 10, 98);

  updateAmbientPetals();
  updateSparkles();
  updateDecayingPetals();

  if (currentState === GAME_STATE.TITLE) {
    drawTitlePage();
  } else if (currentState ===
  GAME_STATE.ANIMATING) {
    drawAnimatingTransition();
  } else if (currentState ===
  GAME_STATE.PLAYING || currentState ===
  GAME_STATE.PAUSED) {
    drawGamePlay();
    if (currentState === GAME_STATE.PAUSED)
    drawPauseOverlay();
  }

  pop();
  shakeAmount *= 0.7;
}
```

```
function mousePressed() {
  if (currentState === GAME_STATE.TITLE)
  handleStartAction();
}

function drawGamePlay() {
  for (let p of permanentGroundPetals) p.display(this);
  drawAmbientPetals(this);
  for (let k of keys) k.drawHighlight(this);

  if (currentState === GAME_STATE.PLAYING) {
    if (frameCount % spawnRate === 0) {
      let l = floor(random(4));
      notes.push(new Note(lanes[l], l));
    }
    for (let k of keys) k.update();
  }

  for (let i = notes.length - 1; i >= 0; i--) {
    let n = notes[i];
    if (currentState === GAME_STATE.PLAYING) n.update();
    n.display(this);
    if (n.isLong && n.isHolding && currentState ===
    GAME_STATE.PLAYING) {
      if (frameCount % 5 === 0) {
        keys[n.laneIndex].burst(n.col, 2);
        if (frameCount % 15 === 0) combo++;
      }
    }
    if (n.isDead) notes.splice(i, 1);
  }

  if (currentState === GAME_STATE.PLAYING) {
    for (let i = ripples.length - 1; i >= 0; i--) {
    ripples[i].update(); if (ripples[i].isDead)
    ripples.splice(i, 1); }
    for (let i = floatingTexts.length - 1; i >= 0; i--) {
    floatingTexts[i].update(); if
    (floatingTexts[i].isDead) floatingTexts.splice(i, 1);
    }
  }
```

```
class Note {
  constructor(x, laneIndex) {
    this.x = x; this.laneIndex = laneIndex; this.y = -50;
    this.col = MACARON_PALETTE[laneIndex];
    this.isLong = random() > 0.7;
    this.initialLen = this.isLong ? random(150, 350) : 0;
    this.len = this.initialLen;
    this.isHit = false; this.isHolding = false;
    this.isDead = false;
    this.rot = random(TWO_PI);
  }
  update() {
    this.y += noteSpeed; this.rot += 0.02;
    if (this.isLong && this.isHolding) {
      this.y = judgeY; this.len -= noteSpeed * 1.2;
      if (this.len <= 0) {
        floatingTexts.push(new JudgementText("Perfect!",
        this.x, judgeY - 40));
        ripples.push(new GrowthRipple(this.x, judgeY));
        this.isDead = true;
      }
    }
    if (!this.isHit && this.y > judgeY + 60) this.miss();
    else if (this.isLong && this.isHit && !this.isHolding
    && this.len > 0) this.miss();
  }
  miss() {
    if (this.isDead) return;
    floatingTexts.push(new JudgementText("Miss", this.x,
    judgeY));
    combo = 0; this.shatter(); this.isDead = true;
  }
  display(target) {
    target.push();
    if (this.isLong && this.len > 0) {
      target.noStroke();
      target.fill(this.col.h, this.col.s, this.col.b,
      this.isHolding ? 60 : 30);
      target.rect(this.x - 12, this.y - this.len, 24,
      this.len, 12);
    }
```

```
  drawHighlight(target) {
    if (this.isPressed || this.highlightAlpha > 0) {
      let alpha = this.isPressed ? 40 :
      this.highlightAlpha;
      target.push(); target.noStroke();
      for(let i=0; i<6; i++) { let w = 70 + i * 5;
      target.fill(0, 0, 100, (alpha / (i + 1)));
      target.rect(this.x - w/2, 0, w, this.y); }
      target.pop();
    }
  }
  display(target) {
    target.noStroke();
    for (let p of this.particles) { target.fill(p.col.h,
    p.col.s, p.col.b, p.alpha); target.push();
    target.translate(p.x, p.y); target.rotate(p.alpha *
    0.1); target.ellipse(0, 0, p.size, p.size * 0.5);
    target.pop(); }
    target.push(); target.translate(this.x, this.y);
    let s = this.isPressed ? 65 : 80;
    target.fill(this.col.h, this.col.s, this.isPressed ?
    100 : this.col.b);
    target.ellipse(0, 0, s, s * 0.7);
    target.fill(0, 0, 100, 40); target.ellipse(0, -s*0.1,
    s * 0.6, s * 0.3);
    target.pop();
  }
  burst(noteColor, count) {
    let burstCol = noteColor || this.col;
    for (let i = 0; i < count; i++) {
      let angle = random(TWO_PI); let spd = random(2, 6);
      this.particles.push({ x: this.x, y: this.y, vx:
      cos(angle)*spd, vy: sin(angle)*spd, alpha: 100,
      size: random(8, 15), col: burstCol });
      if (count > 5) decayingBurstPetals.push(new
      DecayingPetal(this.x + random(-50, 50), height -
      random(5, 50), burstCol));
    }
```

Appendix: Week 12,13 – Final Design

```
function drawAnimatingTransition() {
  gameGraphics.background(globalHueShift, 10, 98);
  for (let p of permanentGroundPetals)
  p.display(gameGraphics);
  drawGamePlay();
  doorOffset += doorSpeed;
  image(gameGraphics, -doorOffset, 0, width/2, height, 0
  0, width/2, height);
  image(gameGraphics, width/2 + doorOffset, 0, width/2,
  height, width/2, 0, width/2, height);
  if (doorOffset > width/2) currentState =
  GAME_STATE.PLAYING;
}

function initializeGameLogic() {
  notes = []; keys = []; permanentGroundPetals = [];
  decayingBurstPetals = []; floatingTexts = []; sparkles
  = []; ripples = []; judgeY = height * 0.75;
  let laneWidth = width / 5; lanes = [width/2 -
  laneWidth*1.5, width/2 - laneWidth*0.5, width/2 +
  laneWidth*0.5, width/2 + laneWidth*1.5];
  for (let i = 0; i < 4; i++) keys.push(new
  MacaronKey(lanes[i], judgeY, MACARON_PALETTE[i]));
  for (let i=0; i<100; i++)
  permanentGroundPetals.push(new
  GroundPetal(random(width), random(height-100, height),
  random(MACARON_PALETTE)));
}

function getLaneFromKeyCode(code) {
  if (code === LEFT_ARROW) return 0; if (code ===
  UP_ARROW) return 1;
  if (code === DOWN_ARROW) return 2; if (code ===
  RIGHT_ARROW) return 3;
  return -1;
}
```

```
class GroundPetal { constructor(x, y, col) { this.x
this.y = y; this.size = random(10, 20); this.angle
random(TWO_PI); this.col = col; this.alpha = 60; }
display(target) { target.push(); target.translate(t
this.y); target.rotate(this.angle); target.noStroke
target.fill(this.col.h, this.col.s - 5, this.col.b,
this.alpha); target.ellipse(0, 0, this.size, this.s
0.7); target.pop(); } }

class DecayingPetal { constructor(x, y, col) { this
x; this.y = y; this.size = random(8, 15); this.angl
random(TWO_PI); this.col = col; this.alpha = 100;
this.life = 150 + random(50); this.isDead = false;
update() { this.life--; if (this.life < 80) this.al
2; if (this.alpha <= 0 || this.life <= 0) this.isDe
true; } display(target) { target.push();
target.translate(this.x, this.y);
target.rotate(this.angle); target.noStroke();
target.fill(this.col.h, this.col.s - 5, this.col.b,
this.alpha); target.ellipse(0, 0, this.size, this.s
0.7); target.pop(); } }

class Sparkle { constructor() { this.x = random(wid
this.y = random(height); this.alpha = 0; this.maxAl
random(40, 80); this.fadeStep = random(1, 2);
this.growing = true; this.size = random(2, 4);
this.isDead = false; this.hue = globalHueShift; }
update() { if (this.growing) { this.alpha += 4; if
(this.alpha >= this.maxAlpha) this.growing = false;
else { this.alpha -= this.fadeStep; if (this.alpha
this.isDead = true; } } display(target) { target.pu
target.noStroke(); target.fill(this.hue, 5, 100,
this.alpha); target.ellipse(this.x, this.y, this.si
target.rectMode(CENTER); target.rect(this.x, this.y
this.size * 3, 0.5); target.rect(this.x, this.y, 0.
this.size * 3); target.pop(); } }
```

```
class JudgementText { constructor(content, x, y) {
this.content = content; this.x = x; this.y = y;
this.alpha = 100; this.scale = 0.5; this.upOffset =
this.isDead = false; this.col = (this.content === "
? { h: 0, s: 0, b: 55 } : { h: 48, s: 85, b: 100 };
update() { this.upOffset -= 1.6; this.alpha -= 2.2;
(this.scale < 1.1) this.scale += 0.1; if (this.alph
0) this.isDead = true; } display(target) { target.p
target.translate(this.x, this.y + this.upOffset);
target.scale(this.scale); target.textAlign(CENTER,
CENTER); if (this.content !== "Miss") { target.stro
0, 100, this.alpha * 0.7); target.strokeWeight(3);
target.fill(this.col.h, this.col.s, this.col.b,
this.alpha); target.textSize(28); target.textStyle(
target.text(this.content, 0, 0); target.pop(); } }

function updateSparkles() { if (frameCount % 8 ===
sparkles.push(new Sparkle()); for (let i =
sparkles.length - 1; i >= 0; i--) { sparkles[i].upd
sparkles[i].display(this); if (sparkles[i].isDead)
sparkles.splice(i, 1); } }

function updateDecayingPetals() { for (let i =
decayingBurstPetals.length - 1; i >= 0; i--) {
decayingBurstPetals[i].update(); if
(decayingBurstPetals[i].isDead)
decayingBurstPetals.splice(i, 1); } }

function updateAmbientPetals() { if (frameCount % 6
&& ambientPetals.length < 80) ambientPetals.push(ne
AmbientPetal(random(width), -20)); for (let i =
ambientPetals.length - 1; i >= 0; i--) {
ambientPetals[i].update(); if (ambientPetals[i].y >
height || ambientPetals[i].alpha <= 0)
ambientPetals.splice(i, 1); } }
```

```
unction updateAmbientPetals() { if (frameCount % 6 === (
& ambientPetals.length < 80) ambientPetals.push(new
mbientPetal(random(width), -20)); for (let i =
mbientPetals.length - 1; i >= 0; i--) {
mbientPetals[i].update(); if (ambientPetals[i].y >
eight || ambientPetals[i].alpha <= 0)
mbientPetals.splice(i, 1); } }

lass AmbientPetal { constructor(x, y) { this.x = x;
his.y = y; this.size = random(8, 18); this.speed =
andom(1, 3); this.rot = random(TWO_PI); this.swing =
andom(0.5, 2); this.col = random(MACARON_PALETTE);
his.alpha = 80; } update() { this.y += this.speed;
his.x += sin(frameCount * 0.02) * this.swing; if (this.y
 height * 0.7) this.alpha -= 1.5; } display(target) {
arget.push(); target.translate(this.x, this.y);
arget.rotate(this.rot); target.noStroke();
arget.fill(this.col.h, this.col.s - 10, this.col.b,
his.alpha); target.ellipse(0, 0, this.size, this.size *
.6); target.pop(); } }

unction drawCombo() { push(); translate(width/2, 100);
extAlign(CENTER, CENTER); noStroke(); let comboHue =
globalHueShift + 40) % 360; fill(comboHue, 20, 80, 50);
extSize(18); text("COMBO", 0, -35); let s = 1 +
in(frameCount * 0.1) * 0.05; scale(s); fill(0, 0, 100,
0); textSize(64); text(combo, 2, 2); fill(comboHue, 50,
0); textStyle(BOLD); text(combo, 0, 0); pop(); }

unction drawAmbientPetals(target) { for (let p of
mbientPetals) p.display(target); }

unction windowResized() { resizeCanvas(windowWidth,
indowHeight); gameGraphics = createGraphics(windowWidth
indowHeight); gameGraphics.colorMode(HSB, 360, 100, 100
00); initializeGameLogic(); }
```

THANK YOU